Syntax_1

Context-free grammars /on whiteboard/:

- Let's create a context-free grammar to recognize&check the syntax of a list like this:

  [apple; pear; plum]

  Terminal symbols of the grammar are:
  o (open bracket), *e* (element), *s* (semicolon), *c* (closing bracket)

  So the upper list should be recognized as: *oesesec*
  Other valid worlds are: *oc*, o*e*c, *oesec* etc.

- Online tool to try/test/play with:
  https://web.stanford.edu/class/archive/cs/cs103/cs103.1156/tools/cfg/
  (https://web.stanford.edu/class/archive/cs/cs103/cs103.1156/tools/cfg/)

- Let's create a context free grammar to recognize C sytle function declarations:

  char is_part(string s, int index);
  int total_sum(int x, int y);

  Terminal symbols:
  id (identifier), *op* (open parenthesis), *cp* (closing parenthesis), *co* (comma), *se* (semicolon)

  The above example should be recognized as *id id op id id co id id cp se id id op id id co id id cp se*

- Let's create a grammar to recognize a simple block-based syntax:

  begin
  skip
  begin end
  end
  begin
  begin skip end
  skip skip skip
  end

  Terminal symbols: *b* (begin), *e* (end), *s* (skip)

The above example should be recognized as *bsbeebbsessse*
Other valid sequences: *ε*, *s*, *ss*, *be*, *sss*, *bse*, *sbe*, *bes*, ssss, *bebe* etc.

- First create a simple lexical analyzer (a few regexps for variables, number literals, keywords, assignment operator, whitespace) to recognize the below example as a sequence of words. Then create a context-free grammar to describe the syntax of this simple language.

  !x1 := 5
  if !x1 odd then !2y := 4
  if !2y even then !x1 := 2

  - Extension 1: the right-hand side of the assignment operator should handle variable names as well (not just number literals)
  - Extension 2: new operators for addition and subtraction, and extend the grammar with new rules to recognize arbitrarily complex expressions.
  - Extension 3: multiple instructions after "then", basically add support for code blocks
  - Extension 4: add a while loop construct to the language